



《人工智能综合设计》—检索排序

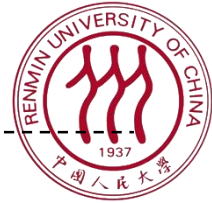


人工智能综合设计 教研组



今日目标

- 改进倒排索引，保存词频信息和文档频率信息
- 基于改进后的倒排索引实现基于tf-idf加权的向量空间模型



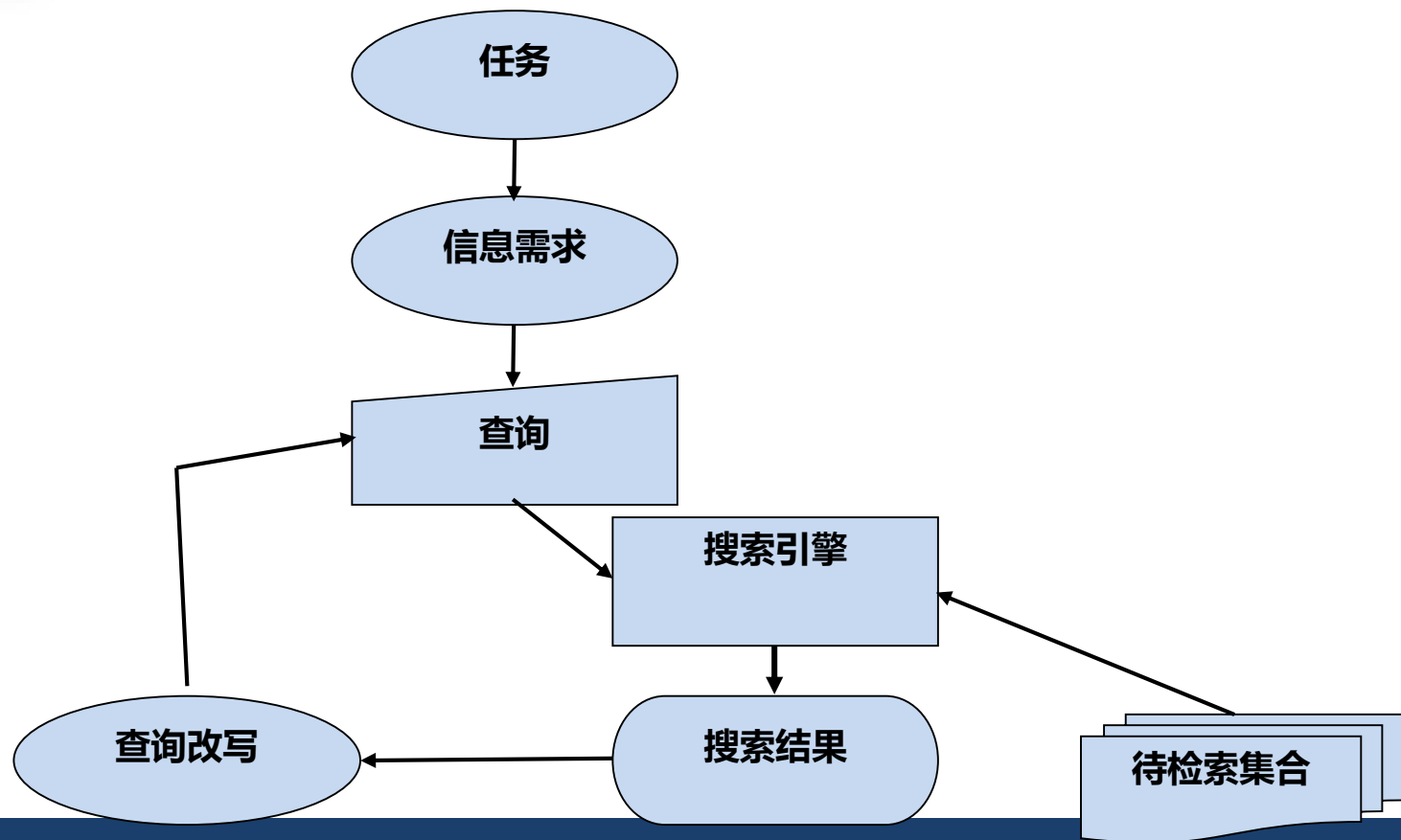
提纲



人工智能综合设计03 信息检索和倒排索引

- ☐ 排序检索
- ☐ 词频加权
- ☐ (逆) 文档频率加权
- ☐ Tf-idf加权
- ☐ 向量空间模型

搜索的过程





排序检索

- 目前为止，我们基于倒排索引构造了一个支持布尔查询的搜索系统
 - AND/OR
 - 一个文档可以与一个查询**匹配或不匹配**
 - 与查询匹配的文档可能很多
- 对于**专业用户**来说，这样一个系统是有效的
 - 用户对自身信息需求和待查询集合有充分的了解
 - 知道如何用布尔查询表达信息需求
 - 专利检索、法律检索、文献调研
- 但对于**普通用户**：
 - 普通用户不会或不愿意组织和提交复杂的布尔查询
 - 大部分不希望浏览大量的与查询匹配的文档



布尔检索的问题

- 布尔查询常常会返回：
 - 过少的匹配结果
 - 过多的匹配结果
- 需要通过不断修改查询，来控制返回结果的数量：
 - AND查询：返回更少的结果
 - OR查询：返回更多的结果



排序检索模型

- 与返回一个文档**集合**的**布尔检索模型**不同，**排序检索模型**可以返回一个**有序**的结果列表
- 同时，用户不需要使用AND和OR等逻辑运算符组织查询，而可以使用自然语言进行查询



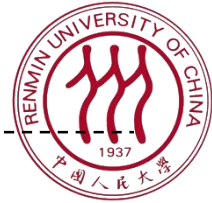
基于文档得分的排序检索模型

- 我们希望将最有可能对用户完成当前任务有用的文档放在排序中靠前的位置
- 使用一个打分函数,给每个文档进行打分:
 - $f(d,q)$
 - 取值范围可以在 $[0, 1]$ 之间
 - 分数越大,说明文档与查询之间的匹配程度越高,文档与查询越**相关**



一个最简单的打分函数

- 假设查询 q 只能包含一个词
- 如果查询 q 没有在文档 d 中出现, 则文档 d 的得分为0
 - $f(q, d)=0$
- 查询 q 在文档 d 中出现的次数越多, 文档 d 的得分越高
 - $f(q, d)$ 是一个 q 在 d 中出现频率 (词频) 的单调函数



提纲



人工智能综合设计03 信息检索和倒排索引

- ☐ 排序检索
- ☐ 词频加权
- ☐ (逆) 文档频率加权
- ☐ Tf-idf加权
- ☐ 向量空间模型



回顾：词-文档矩阵

	文继荣	窦志成	卢志武	徐君	魏哲巍	赵鑫	宋睿华	严睿	许洪腾	苏冰	王子贺	沈蔚然	胡迪	刘勇	毛佳昕	陈旭
信息检索	1	1	0	1	0	1	1	1	0	0	0	0	0	0	1	1
搜索	1	1	1	1	1	0	0	0	0	0	0	0	0	1	1	0
推荐	0	0	0	1	0	1	1	0	0	0	0	0	0	0	0	1
数据库	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
语言	0	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0
视觉	0	0	1	0	0	0	0	0	0	1	0	0	1	0	0	0
对话	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0
数据挖掘	1	1	0	1	0	1	0	0	0	0	0	0	0	1	0	0
经济学	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
博弈论	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0

- 用一个长度为 V 的向量来表示一个文档，向量中元素 $\in \{0, 1\}$

保存词在文档中出现次数的矩阵

- 考虑词在文档中出现的次数
 - 长度为V的向量，向量中元素 $\in \mathbb{N}$

	文继荣	窦志成	卢志武	徐君	赵鑫	魏哲巍	宋睿华	严睿	祁琦	孙浩	苏冰	许洪腾	刘勇	王子贺	毛佳昕	陈旭	胡迪	沈蔚然
信息检索	2	10	0	7	3	0	2	2	0	0	0	0	0	0	5	1	0	0
搜索	3	4	1	4	0	2	0	0	0	0	0	0	1	0	4	0	0	0
推荐	0	0	0	1	5	0	1	0	0	0	0	0	0	0	0	6	0	0
数据库	1	0	0	0	0	4	0	0	0	0	0	0	0	0	0	0	0	0
语言	0	1	1	0	2	0	0	0	0	0	0	0	0	0	0	0	0	0
视觉	0	0	3	0	0	0	0	0	0	0	3	0	0	0	0	0	1	0
对话	0	0	0	0	0	0	1	2	0	0	0	0	0	0	0	0	0	0
数据挖掘	2	1	0	1	1	0	0	0	0	0	0	0	1	0	0	0	0	0
经济学	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
博弈论	0	0	0	0	0	0	0	0	3	0	0	0	0	3	0	0	0	2

词袋 (bag of words)模型

- 上述用长度为 V 的向量来表示一个文档的方式忽略了词在文档中出现的顺序
- “葡萄比苹果甜” 和 “苹果比葡萄甜” 的向量表示是一样的
- 这种表示文档的方式有个形象的名称：词袋模型





词频 (Term frequency)

- 我们用 $tf_{t,d}$ 来表示词 t 在文档 d 中出现的次数 (词频)
- 可以利用 tf 来计算查询和文档之间的匹配分数
- 然而, 之间利用词频计算分数会存在问题:
 - 一个 tf 为10的文档 a 会比一个 tf 为1的文档 b 与查询更相关
 - 但 a 并不一定比 b 相关恰好10倍
- 相关性与 tf 之间存在**单调**的关系, 但并不一定是**线性**的关系
 - $tf_{t,d1} > tf_{t,d2}$ 则 $f(t, d1) > f(t, d2)$
 - $tf_{t,d1} = k * tf_{t,d2}$, $f(t, d1) \neq k * f(t, d2)$

对数频率

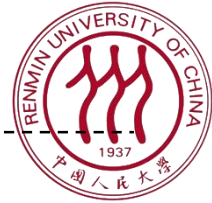
- 词t在文档d中出现的对数频率（对数权重）为：

$$w_{t,d} = \begin{cases} 1 + \log_{10} \text{tf}_{t,d}, & \text{if } \text{tf}_{t,d} > 0 \\ 0, & \text{otherwise} \end{cases}$$

- 对包含多个词的查询进行打分：

- $$\text{Score} = \sum_{t \in q \cap d} (1 + \log \text{tf}_{t,d})$$

- 若没有一个查询词在文档中出现，则score为0



提纲



人工智能综合设计03
信息检索和倒排索引

- □ 排序检索
- □ 词频加权
- □ (逆) 文档频率加权
- □ Tf-idf加权
- 向量空间模型



文档频率

- 相较于较为常见的词，较为罕见的词可能包含更多的信息
 - 考虑停用词(stop words)
- 如果有一个较为罕见的词出现在了文档中（例如：信息检索）
- 那么一个包含该词的文档，就有很大的可能性和“信息检索”这个查询相关
- 因此，我们在设计打分函数时，应该给这些词更高的权重！



idf加权

- 用 df_t 表示词 t 在文档中出现的次数：包含该词的文档有多少个
 - df_t 和词 t 包含的信息多少呈负相关关系
 - $df_t \leq N$: N 为集合中文档的数量
- 我们定义词的逆文档概率 (inverse document frequency, idf) 为：

$$idf_t = \log_{10} (N/df_t)$$

- 与 tf 类似，我们使用对数函数来防止 idf 带来特别极端的影响

idf计算举例，假设 $N=1,000,000$

term	df_t	idf_t
calpurnia	1	6
animal	100	4
sunday	1,000	3
fly	10,000	2
under	100,000	1
the	1,000,000	0

$$idf_t = \log_{10} (N/df_t)$$

- 对于不同待检索集合，一个词的idf值是不同的



idf在排序中的效果

- 问题：idf在只包含一个词的查询中有什么用吗？
 - 例如：iphone



idf在排序中的效果

- 问题：idf在只包含一个词的查询中有什么用吗？
 - 例如：iphone
- 回答：idf在只有一个词的查询中不起作用
 - idf只会影响包含两个或以上词的查询的排序结果
 - 对于查询：iphone 手机
 - idf加权会使iphone这个词在计算排序分数时相对于更常见的词（手机）有更大的权重

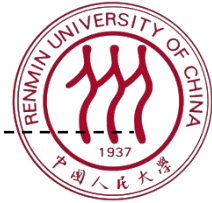


集合频率vs文档频率

- 集合频率 (collection frequency, cf): 词在整个待检索集中出现的次数
- 一个词在一个文档中出现多次
 - df: 只算出现一次
 - cf: 算出现多次
- cf和df比较的例子:

Word	Collection frequency	Document frequency
<i>insurance</i>	10440	3997
<i>try</i>	10422	8760

- 应该在设计打分模型时给哪个词更高的权重?



提纲



人工智能综合设计03 信息检索和倒排索引

- ☐ 排序检索
- ☐ 词频加权
- ☐ (逆) 文档频率加权
- ☐ Tf-idf加权
- ☐ 向量空间模型

Tf-idf加权

- Tf-idf加权：用tf权重和idf权重的乘积计算匹配分数

$$w_{t,d} = (1 + \log \text{tf}_{t,d}) \times \log_{10}(N / \text{df}_t)$$

- 信息检索领域最著名的评分方式
- 一个词在文档中出现的次数越多
- 这个词在待检索集合中越罕见
- 那么这个词对相关性分数的贡献就越大



最终文档的相关性分数

$$\text{Score}(q, d) = \sum_{t \in q \cap d} \text{tf.idf}_{t,d}$$



01值->出现次数->权重矩阵

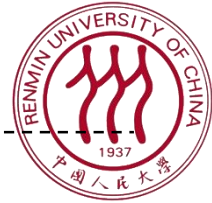
	文继荣	窦志成	卢志武	徐君	赵鑫	魏哲巍	宋睿华	严睿	祁琦	孙浩	苏冰	许洪腾	刘勇	王子贺	毛佳昕	陈旭	胡迪	沈蔚然	idf
信息检索	2	10	0	7	3	0	2	2	0	0	0	0	0	0	5	1	0	0	信息检索 0.35
搜索	3	4	1	4	0	2	0	0	0	0	0	0	1	0	4	0	0	0	搜索 0.41
推荐	0	0	0	1	5	0	1	0	0	0	0	0	0	0	0	6	0	0	推荐 0.65
数据库	1	0	0	0	0	4	0	0	0	0	0	0	0	0	0	0	0	0	数据库 0.95
语言	0	1	1	0	2	0	0	0	0	0	0	0	0	0	0	0	0	0	语言 0.78
视觉	0	0	3	0	0	0	0	0	0	0	3	0	0	0	0	0	1	0	视觉 0.78
对话	0	0	0	0	0	0	1	2	0	0	0	0	0	0	0	0	0	0	对话 0.95
数据挖掘	2	1	0	1	1	0	0	0	0	0	0	0	1	0	0	0	0	0	数据挖掘 0.56
经济学	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	经济学 1.26
博弈论	0	0	0	0	0	0	0	0	3	0	0	0	0	3	0	0	0	2	博弈论 0.95



01值->出现次数->权重矩阵



	文继荣	窦志成	卢志武	徐君	赵鑫	魏哲巍	宋睿华	严睿	祁琦	孙浩	苏冰	许洪腾	刘勇	王子贺	毛佳昕	陈旭	胡迪	沈蔚然
信息检索	0.46	0.70	0.00	0.65	0.52	0.00	0.46	0.46	0.00	0.00	0.00	0.00	0.00	0.00	0.60	0.35	0.00	0.00
搜索	0.61	0.66	0.41	0.66	0.00	0.53	0.00	0.00	0.00	0.00	0.00	0.00	0.41	0.00	0.66	0.00	0.00	0.00
推荐	0.00	0.00	0.00	0.65	1.11	0.00	0.65	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.16	0.00	0.00
数据库	0.95	0.00	0.00	0.00	0.00	1.53	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
语言	0.00	0.78	0.78	0.00	1.01	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
视觉	0.00	0.00	1.15	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.15	0.00	0.00	0.00	0.00	0.00	0.78	0.00
对话	0.00	0.00	0.00	0.00	0.00	0.00	0.95	1.24	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
数据挖掘	0.72	0.56	0.00	0.56	0.56	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.56	0.00	0.00	0.00	0.00	0.00
经济学	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.26	0.00	0.00	0.00	0.00
博弈论	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.15	0.00	0.00	0.00	0.00	1.15	0.00	0.00	0.00	1.01



提纲



人工智能综合设计03 信息检索和倒排索引

- □ 排序检索
- □ 词频加权
- □ (逆) 文档频率加权
- □ Tf-idf加权
- 向量空间模型



用向量表示文档

- 现在我们可以用一个V维向量来表示一个文档
 - 词-文档矩阵: 01值
 - 词频矩阵: tf
 - if-idf权重矩阵: $\log(1+tf)*\log(N/df)$
- 每个词是向量空间的一个坐标轴
 - 高维向量空间
- 每个向量均很稀疏 (sparse)
 - 一个向量中非零的元素个数远小于向量维数



用向量来表示查询

- Key idea 1: 用同样的方式来表示查询
 - 将查询也表示为一个高维稀疏向量
- Key idea 2: 用两个向量在向量空间中的近似程度来进行排序
 - 向量的相似度
 - 距离的倒数
- 目标:
 - 解决布尔检索模型只能做出 匹配/不匹配 判断的问题
 - 将与查询更相关的文档排在更靠前的位置



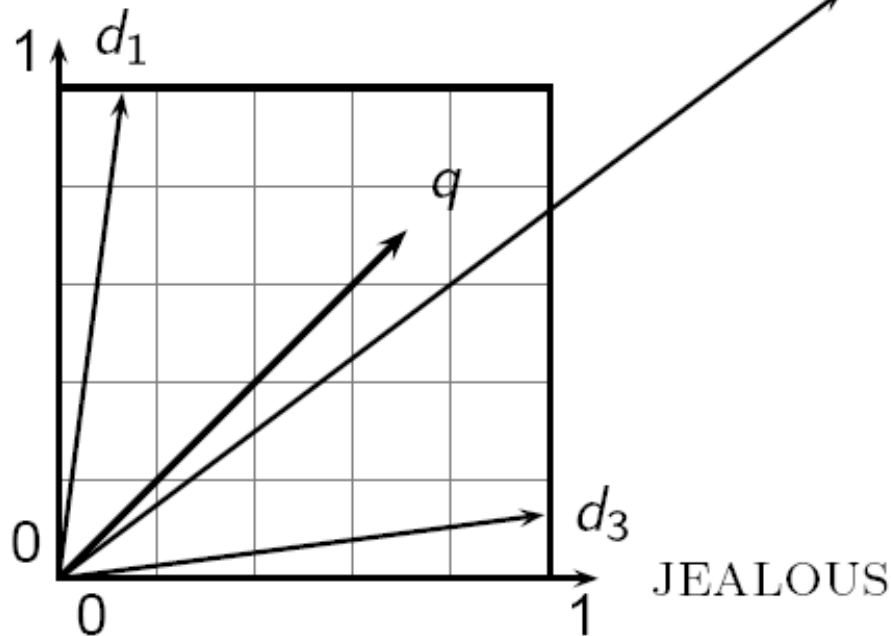
定义两个向量的相似程度

- 第一个想法：
 - 计算两个向量之间的**距离**
- 欧式距离？
- 效果很差
-因为两个模长不同的向量之间的距离很大
 - 查询向量
 - 文档向量

为什么不能使用欧式距离？

- 尽管 q 和 d_2 的词分布更为接近
- 但 q 和 d_2 之间的距离非常大

GOSSIP





使用向量间的夹角

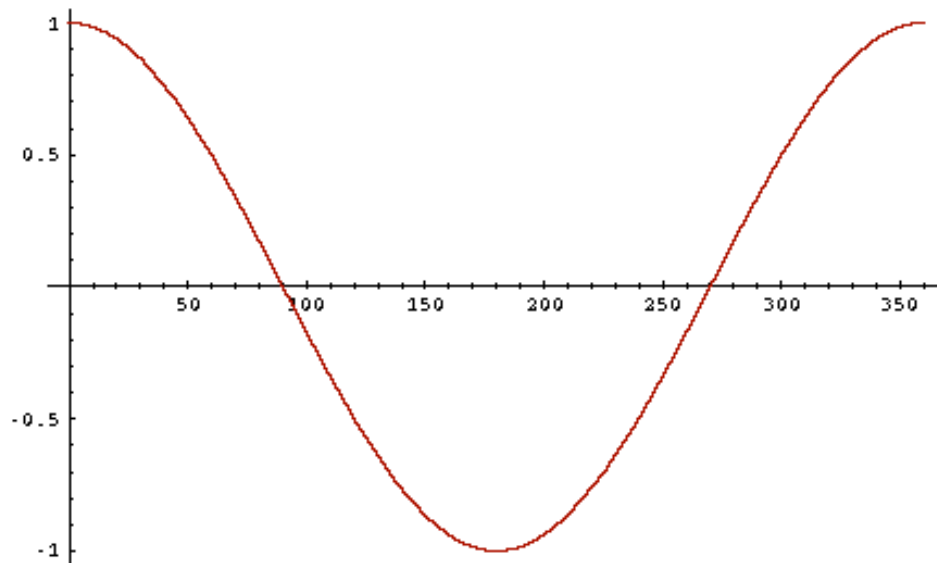
- 考虑这样一个实验：
 - 把文档d的内容复制一份加入到自身的后面得到文档d'
 - 文档d和文档d' 在语义（表达的含义）上是非常相似的
 - d和d' 之间的欧式距离可以很大
 - d和d' 之间的夹角为0，能够表示d和d' 之间语义上的相似性
- Key idea: 按照文档向量与查询向量之间的夹角，从小到大对文档进行排序



夹角和余弦相似度

- 以下两种计算方式是等价的：
 - 将文档按照文档与查询向量间的夹角大小，**从小到大**排序
 - 将文档按照文档与查询向量间的夹角的cosine值，**从大到小**排序
- Cosine函数在 $[0, \pi]$ 区间（ $[0^\circ, 180^\circ]$ ）内单调递减

夹角和余弦相似度



- 我们为什么要计算余弦相似度？
- 我们该如何计算余弦相似度？

长度归一化

- 通过除以模长，可以将一个向量的模长归一化为1

$$\|\vec{x}\|_2 = \sqrt{\sum_i x_i^2}$$

- 将向量转化为单位向量
- 回忆刚才d和d' 的例子
 - 在进行归一化后，会得到完全相同的向量表示
 - 长度不同的文档的权重会变得可比较

查询和文档间的余弦相似度

- 计算Cosine (q, d) :

$$\cos(\vec{q}, \vec{d}) = \frac{\vec{q} \bullet \vec{d}}{|\vec{q}| |\vec{d}|} = \frac{\vec{q}}{|\vec{q}|} \bullet \frac{\vec{d}}{|\vec{d}|} = \frac{\sum_{i=1}^{|V|} q_i d_i}{\sqrt{\sum_{i=1}^{|V|} q_i^2} \sqrt{\sum_{i=1}^{|V|} d_i^2}}$$

- q_i : 第*i*个词在查询中的tf-idf权重
- d_i : 第*i*个词在文档中的tf-idf权重
- 两个tf-idf向量的夹角的余弦值

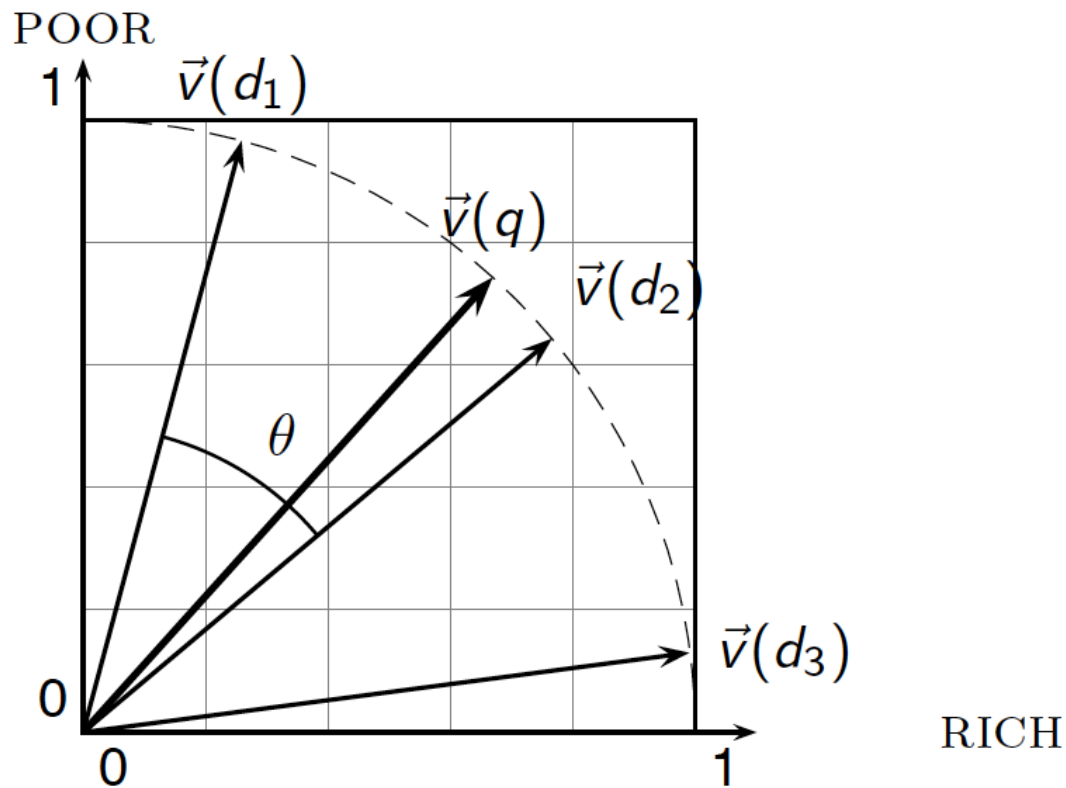


归一化向量的余弦相似度

- 对于归一化后的向量（长度为1的单位向量），余弦相似度就是内积：

$$\cos(\vec{q}, \vec{d}) = \vec{q} \cdot \vec{d} = \sum_{i=1}^{|V|} q_i d_i$$

余弦相似度的示意图





文档间的余弦相似度

- 三位老师个人主页的相似度
 - 为简化计算我们只考虑6个词
 - 在一个6维子空间中计算余弦相似度
 - 不考虑idf权重

	文继荣	窦志成	卢志武
信息检索	2	10	0
搜索	3	4	1
推荐	0	0	0
数据库	1	0	0
语言	0	1	1
视觉	0	0	3

文档间的余弦相似度

- 计算对数词频

	文继荣	窦志成	卢志武
信息检索	1.10	2.40	0.00
搜索	1.39	1.61	0.69
推荐	0.00	0.00	0.00
数据库	0.69	0.00	0.00
语言	0.00	0.69	0.69
视觉	0.00	0.00	1.39

- 长度归一化

	文继荣	窦志成	卢志武
信息检索	0.58	0.81	0.00
搜索	0.73	0.54	0.41
推荐	0.00	0.00	0.00
数据库	0.36	0.00	0.00
语言	0.00	0.23	0.41
视觉	0.00	0.00	0.82

- $\cos(\text{文继荣}, \text{窦志成}) = 0.58 * 0.81 + 0.73 * 0.54 = 0.86$
- $\cos(\text{文继荣}, \text{卢志武}) = 0.30$
- $\cos(\text{窦志成}, \text{卢志武}) = 0.32$

基于倒排索引计算余弦相似度

- 需要在倒排索引中保存**词频信息(tf)**和文档频率信息
 - $df = \text{postings list 的长度}$

```
# 构造倒排索引
# postings_list中每一项为一个Posting类对象
class Posting(object):
    def __init__(self, docid, tf=0):
        self.docid = docid
        self.tf = tf
    def __repr__(self):
        return "<docid: %d, tf: %d>" % (self.docid, self.tf)

# 为了编程实现方便, 我们在每个postings_list开头加上了一个用来标记开头的Posting(-1, 0) (正常docid从0编号)
# 注意这段代码只有在term_docid_pairs已排序的时候才是正确的
from collections import defaultdict
inverted_index = defaultdict(lambda: [Posting(-1, 0)])

for term, docid in term_docid_pairs:
    postings_list = inverted_index[term]
    if docid != postings_list[-1].docid:
        postings_list.append(Posting(docid, 1))
    else:
        postings_list[-1].tf += 1

inverted_index = dict(inverted_index)
```

基于倒排索引计算余弦相似度

- 若要进行长度归一化，则还需要保存文档**向量长度**
 - 注意不是文档包含多少个词
 - 一个不考虑idf，但使用了对数词频计算的例子

```
: # 依次打开每个保存正文信息的文本文件，分词，构造term_docid_pairs, 计算文档长度
term_docid_pairs = []
doc_length = []
from collections import Counter
import numpy as np
for docid, filename in enumerate(collections):
    with open(os.path.join('faculty', filename)) as fin:
        terms = [term for term in jieba.cut(fin.read()) if len(term.strip()) > 1] # 必要的过滤，建议实现一个更加复杂的函数，
        # 构造term_docid_pairs
        for term in terms:
            term_docid_pairs.append((term, docid))
        # 统计tf
        term_counts = np.array(list(Counter(terms).values()))
        log_func = np.vectorize(lambda x: 1.0 + np.log(x) if x > 0 else 0.0)
        log_tf = log_func(term_counts)
        doc_length.append(np.sqrt(np.sum(log_tf**2)))

print(term_docid_pairs[0:5])
print('...')
print(term_docid_pairs[-5:])
print(doc_length)
```

基于倒排索引计算余弦相似度

COSINESCORE(q)

```
1  float Scores[ $N$ ] = 0
2  float Length[ $N$ ]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5      for each pair( $d, tf_{t,d}$ ) in postings list
6      do Scores[ $d$ ] +=  $w_{t,d} \times w_{t,q}$ 
7  Read the array Length
8  for each  $d$ 
9  do Scores[ $d$ ] = Scores[ $d$ ] / Length[ $d$ ]
10 return Top  $K$  components of Scores[]
```

基于倒排索引计算余弦相似度

- 一个没有考虑id加权的Python实现

```
1 def get_postings_list(inverted_index, query_term):
2     try:
3         return inverted_index[query_term][1:]
4     except KeyError:
5         return []
6
7 log_func = np.vectorize(lambda x: 1.0 + np.log10(x) if x > 0 else 0.0)
8
9
10 def cosine_scores(inverted_index, doc_length, query, k=3):
11     scores = defaultdict(lambda: 0.0) # 保存分数
12     query_terms = Counter(term for term in jieba.cut(query)) # 对查询进行分词
13     for q in query_terms:
14         w_tq = log_func(query_terms[q])
15         postings_list = get_postings_list(inverted_index, q)
16         for posting in postings_list:
17             w_td = log_func(posting.tf)
18             scores[posting.docid] += w_td * w_tq
19     results = [(docid, score / doc_length[docid]) for docid, score in scores.items()]
20     results.sort(key=lambda x: -x[1])
21     return results[0:k]
22
23 def retrieval_by_cosine_sim(inverted_index, collections, query, k=3):
24     top_scores = cosine_scores(inverted_index, doc_length, query, k=k)
25     results = [(collections[docid], score) for docid, score in top_scores]
26     return results
```



向量空间模型总结

- 将查询和文档表示成tf-idf向量
- 通过计算查询向量和文档向量之间的余弦相似度对其相关性进行评分
- 按照余弦相似度从大到小对文档进行排序
- 将top结果（例如：top 10）结果返回给用户

附录：不同的tf-idf加权方式

Term frequency		Document frequency		Normalization	
n (natural)	$tf_{t,d}$	n (no)	1	n (none)	1
l (logarithm)	$1 + \log(tf_{t,d})$	t (idf)	$\log \frac{N}{df_t}$	c (cosine)	$\frac{1}{\sqrt{w_1^2 + w_2^2 + \dots + w_M^2}}$
a (augmented)	$0.5 + \frac{0.5 \times tf_{t,d}}{\max_t(tf_{t,d})}$	p (prob idf)	$\max\{0, \log \frac{N - df_t}{df_t}\}$	u (pivoted unique)	$1/u$
b (boolean)	$\begin{cases} 1 & \text{if } tf_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$			b (byte size)	$1/CharLength^\alpha$, $\alpha < 1$
L (log ave)	$\frac{1 + \log(tf_{t,d})}{1 + \log(\text{ave}_{t \in d}(tf_{t,d}))}$				

谢谢!

